

Veritas AI: Project Report

Veritas AI is a full-stack, enterprise-grade plagiarism and paraphrase detection web application. Here is a comprehensive overview of the project structure, technology stack, and architecture to help you understand it completely.

Technology Stack

The project relies on a modern and robust technology stack, split between the backend (Python) and frontend (React).

Backend

- **Framework**: FastAPI (High performance, async-first Python web framework)
- **Database**: PostgreSQL (Default) with an async SQLite fallback (`data/plagiarism\_checker.db`)
- **ORM**: SQLAlchemy (Async database models & connection pooling)
- **NLP & ML Engine**:
 - `scikit-learn` (TF-IDF vectorization for exact matching)
 - `sentence-transformers` (`all-MiniLM-L6-v2` model for semantic embeddings to detect paraphrasing)
 - `nltk` (Tokenization)
- **PDF Generation**: ReportLab (For generating downloadable PDF reports)
- **Testing**: Pytest

Frontend

- **Framework**: React 19 with TypeScript
- **Build Tool**: Vite
- **Styling**: Vanilla CSS (Obsidian glassmorphism design system)
- **Routing**: React Router (`react-router-dom`)
- **API Client**: Axios (with custom typed hooks like `useApi.ts`)

3-Tier Detection Engine

The core of Veritas AI is its sophisticated 3-tier hybrid detection engine:

1. **Tier 1 (Lexical Fast Filter)**: Uses character n-gram (3-5) TF-IDF vectorization with sublinear term-frequency scaling and cosine similarity. Good for detecting exact copy-pasting.
2. **Tier 2 (Semantic Embeddings)**: Uses dense 384-dimensional vector embeddings to detect paraphrasing, synonym substitution, and idea theft where the words are changed but the meaning is the same.
3. **Tier 3 (Web Search Integration)**: Integrates with the Tavily API to perform live web searches and cross-reference against public internet content.

Architecture & Directory Structure

The repository is logically organized into ``backend``, ``frontend``, and ``data`` directories.

```
```mermaid
graph TD
 Root["d:\plagiarism checker"] --> Backend["backend/"]
 Root --> Frontend["frontend/"]
 Root --> Data["data/"]
 Backend --> BackendApp["app/"]
 BackendApp --> Models["models/ (SQLAlchemy DB)"]
 BackendApp --> Routers["routers/ (API Endpoints)"]
 BackendApp --> Services["services/ (Core Logic & ML)"]
 BackendApp --> Config["core/config.py"]
 Frontend --> FrontendSrc["src/"]
 FrontendSrc --> Components["components/ (React UI)"]
 FrontendSrc --> Hooks["hooks/ (Custom React hooks)"]
 FrontendSrc --> Styles["styles/ (CSS)"]
```
```

1. Backend (``/backend``)

The FastAPI application is cleanly structured using the service-repository pattern:

- `**`app/main.py`**`: The main entry point initializing CORS, routers, and the database.
- `**`app/services/detector.py`**`: Contains the complex logic for the 3-tier detection engine.
- `**`app/services/parser.py`**`: Handles parsing of multiple formats (PDF, DOCX, TXT) using tools like PyMuPDF.
- `**`app/services/chunker.py`**`: Splits large documents into sentence-level chunks to prevent memory overflows.
- `**`app/routers/`**`: Contains the REST endpoints for analysis (``analysis.py``) and reference document management (``documents.py``).

2. Frontend (``/frontend``)

The React application is built for interaction and visual feedback:

- `**`src/App.tsx`**`: Defines the main routing structure (``/``, ``/results/:id``, ``/history``, ``/references``).
- `**`src/components/`**`: Contains key modules like ``FileUpload`` (for ingestion), ``ResultsDashboard`` (for viewing match statistics), and ``ReferenceDocs`` (for managing the corpus).
- `**`src/styles/components.css`**`: Powers the custom UI design which features color-coded highlights for plagiarism (Rose), suspicious (Amber), and cited text (Purple).

3. Data (``/data``)

This directory acts as the local storage location for the SQLite database fallback, uploaded reference materials, generated PDF reports, and temporary processing artifacts.

Smart Features

> [!TIP]

> The engine is designed to minimize false positives by ignoring academic boilerplate and proper citations.

- **Edge Case Handling**: Automatically detects and ignores bracket citations ([1, 2]), parenthetical markers, and quoted passages, marking them as "Cited" instead of plagiarized.
- **Boilerplate Filter**: Utilizes a curated stop-phrase dictionary to ignore common academic phrases (e.g., "in conclusion", "the results show that").
- **Batch Processing**: Capable of processing massive documents (100+ pages) without timing out by using windowed chunking.

How to Run Locally

1. **Backend**:

Navigate to `backend`, activate the virtual environment (`.venv\Scripts\Activate.ps1`), and run `python run.py`. It starts on port `8000`.

2. **Frontend**:

Navigate to `frontend`, install dependencies (if needed), and run `npm run dev`. It starts on port `5173`.